INKLOOM AI

FINAL PROJECT COMPLETION

MASTER IMPLEMENTATION PROMPT

A detailed, copy-ready instruction to complete the existing project, fix the audited gaps, implement every Round 1 feature, and deliver genuine social-media account connections and approved publishing.

USE WITH THE LATEST PROJECT ZIP + ROUND 1 PROPOSAL PDF

This document contains the implementation prompt only. It is designed for a coding assistant or development agent that can inspect and modify the supplied repository, run commands, connect services and test the completed application.

INSTRUCTIONS AND SUCCESS DEFINITION

You are a senior full-stack engineer, solution architect, database engineer, AI engineer, security engineer, DevOps engineer and QA lead. You have been given the CURRENT INKLOOM AI source ZIP, the official 10-page Round 1 proposal PDF and the latest project audit findings.

Your job is to turn the EXISTING INKLOOM AI project into a complete, reliable, runnable final working product. Do not create a separate replacement project. Work inside the supplied repository, preserve the existing UI and useful components, and implement the missing functionality described here. Do not stop after analysis, planning, scaffolding or writing documentation. Make the code changes, run the application, run tests, fix failures and provide evidence.

The Round 1 proposal remains the source of truth for original product requirements and priority. The latest project ZIP is the source of truth for what code currently exists. This prompt specifies the remaining implementation work and the additional social-media publishing requirement requested by the product owner.

A feature is COMPLETE only if its UI action works, the server authenticates and authorizes it, the data is persisted in the real database, errors are handled, and automated tests cover the important behavior. A screen, mock response, localStorage record, simulated OAuth flow or generated fake post ID does not count as a completed feature.

Do not claim live social publishing is verified unless real provider credentials, permissions and a successful provider response are available. Where external credentials are unavailable, finish the integration code and configuration, clearly show “not connected / credentials required”, and provide exact setup steps.

1. CURRENT REPOSITORY — SPECIFIC FINDINGS TO FIX

The current ZIP already contains a substantial React + TypeScript frontend, Express server, service layer, page components, API contract, SQL schema and demo data. Preserve and improve this work.

The current source includes pages/components such as DashboardPage, BrandHubPage, GuidelinesPage, SubmissionPage, ResultsPage, ReviewQueuePage, ApprovalPage, HistoryPage, AuditLogPage, UsersPage, AnalyticsPage, SettingsPage, MySubmissionsPage, SocialPublishingPage, SocialPublishModal and CommentsThread. Reuse them where sound; add missing screens only where needed.

The latest audit identified these concrete problems. Verify each against the supplied current source before editing, then fix it:

1. `server.ts` currently imports the shared `store` from `src/services/store.ts`. It reads and writes the in-memory/demo store rather than using the PostgreSQL schema as the live persistence layer.

2. `getAuthContext()` in `server.ts` treats a token as a user ID, email or role and permits `X-Demo-User` to select a persona. This is not authentication. Replace it with real authentication and server-side authorization.

3. The current route set is incomplete compared with `docs/API_CONTRACT.md`. The server has read-focused routes for users, brand profiles, guidelines, facts, terms, assets, submissions and review queue, plus analysis, eval, social demo routes and comments. Implement the missing create/update/decision/version/audit/analytics/auth routes.

4. `src/api/client.ts` reads local browser storage and sends `X-Demo-User`. It also sends an `X-Gemini-Key` header during analysis. Remove these insecure production behaviors.

5. `src/services/store.ts` uses localStorage and demo state for shared records, including social accounts, publishing jobs, comments and other business data. Replace shared business persistence with database-backed API calls.

6. `src/services/aiPipeline.ts` and its integration must not depend on a client-supplied API key. All AI provider calls and secrets must remain server-side.

7. `db/schema.sql` defines a useful relational starting point, but a schema file is not a connected database. Add a database driver/client, migrations, runtime repository layer and verified database health checks.

8. `SocialPublishingPage.tsx` currently offers simulated connection behavior. The server's `POST /api/social/accounts/connect` accepts manually supplied display name/handle and generates an external account ID, granted scopes and “connected” state. Replace this with official provider OAuth.

9. Current social publish/schedule routes must not manufacture success. The social UI must not treat local/demo records as actual provider posts.

10. `SocialPublishModal.tsx` currently starts from the source submission's latest version and a hard-coded sample hashtag set. Implement saved channel-specific variants and brand-aware editable previews instead.

11. The current social approval check is not sufficient. A source submission's approval must not automatically approve a modified channel-specific caption or asset. Remove any default `is_approved: true` behavior.

12. `CommentsThread.tsx` calls API methods, but comments must be stored and retrieved from the database, scoped to a submission and exact version, and authorized by role.

13. The API contract and README contain demo-oriented instructions and credentials/persona switching. Keep demo seed data for local development only, disable persona switching in production and clearly label all seeded content as fictional.

14. The current package dependencies do not include a PostgreSQL client, job queue/worker framework or production authentication implementation. Select and add appropriate maintained dependencies, documenting the choice.

15. The current project uses one Express server. The Round 1 proposal planned a separate Python FastAPI AI service. Implement the planned service boundary where practical, or document a justified equivalent architecture that keeps AI isolated behind the backend and never exposes provider secrets to the browser.

2. ROUND 1 REQUIREMENTS — COMPLETE THE ORIGINAL PRODUCT

Implement all 15 features from the Round 1 proposal. Respect its original priority: features 1–8 were MUST, 9–13 SHOULD, and 14–15 LATER. The final product should implement the full requested set with honest limits for advanced checks.

2.1 Brand Knowledge Hub (MUST)

• A brand profile stores organisation, brand name, positioning, audience, voice notes, values, tone attributes, prohibited tone, channel guidance and palette.

• Admin can create, read, update and archive profiles.

• Store approved facts, preferred terms, avoided terms, guidelines and approved assets linked to the correct brand.

• Every record has an owner, timestamps, active/archived state and source metadata where applicable.

• Enforce organisation and brand-level access on every query.

2.2 Guideline Manager (MUST)

• Create, read, update, deactivate and version individual rules.

• Each rule has stable code, title, category, severity, rule text, applicable content types/channels, good/bad examples and active state.

• Editing a rule creates a new version; do not overwrite the old rule used in past analysis.

• Analysis reports must cite the exact guideline version used.

• Add a paste/import workflow if supported by the current design, but require admin review before imported rules become active.

2.3 Content Submission (MUST)

• Creators can create drafts, choose brand and content type, enter content and attach supported assets.

• Save version 1 at creation and persist every later version.

• Validate required fields, content size, file types and brand permissions.

• Support list/detail/search/filter and pagination.

• Provide clear draft, analysis, correction, review, changes-requested, approved, rejected and published states as appropriate.

2.4 AI Brand Voice Analyzer (MUST)

• Compare the exact submitted version with active approved brand voice and guideline references.

• Return findings with category, severity, confidence/uncertainty, exact excerpt, location where possible, explanation, reference code and exact reference version.

• Identify on-brand strengths where supported.

• Never invent a rule, fact or excerpt.

• Keep model/provider, prompt version, brand context version, guideline versions and analysis timestamp.

2.5 AI Correction Assistant (MUST)

• Suggest the smallest change that fixes each finding while preserving author intent and factual meaning.

• Show original and proposed text, reason and affected finding.

• User can accept, edit or dismiss each suggestion; require a reason for dismissing high-severity findings.

• Never silently replace source content.

• Saving any accepted correction creates a new immutable content version and triggers analysis for that new version.

2.6 Review and Approval (MUST)

• Reviewers see a real queue sorted by severity and age.

• Reviewer can inspect exact content version, report, references, comments and diff.

• Reviewer can approve, reject or request changes with a reason.

• Only authorized human reviewers/admins can make final decisions.

• Prevent self-approval according to the Round 1 rule.

• Approval attaches to an exact immutable version ID, never merely to the submission ID.

• Any changed version returns to analysis and review.

2.7 Version History (MUST)

• Every submitted, edited, corrected, reviewed, approved and published version is immutable.

• Store version number, content, assets, author, timestamp, source version and change summary.

• Show version timeline and side-by-side or word-level diff.

• Preserve the exact version approved and the exact version published.

2.8 Role-Based Access (MUST)

• Implement Admin, Creator, Reviewer and Viewer roles.

• Admin manages users, brands, guidelines, facts, terms and assets.

• Creator manages permitted drafts and suggestions, and submits for review.

• Reviewer manages assigned/permitted review decisions and comments.

• Viewer has read-only access to permitted reports/analytics/audit information.

• Enforce permissions on the server for every endpoint and record.

• Frontend hiding is usability only, never the security boundary.

2.9 Terminology Checker (SHOULD)

• Implement deterministic code-based matching for preferred/avoided terms.

• Support case sensitivity, word boundaries, plural/variant handling where configured, content/channel scope and replacement suggestions.

• Return exact excerpts and terminology reference codes.

2.10 Approved Fact Check (SHOULD)

• Compare factual claims with approved facts and source references.

• Distinguish verified match, verified conflict, not covered/unsupported and needs-human-check.

• Show the matched fact code, exact fact statement, source reference and validity/expiry.

• Expired facts must not be treated as current approval.

• Absence of evidence is not proof that a claim is false.

2.11 Unsupported Claim Detection (SHOULD)

• Flag claims such as “best”, “number one”, “guaranteed” or “unlimited” when the approved brand knowledge does not support them.

• Label them unsupported or needs human verification, not false.

• Show the exact excerpt and explain what evidence would be needed.

• Do not invent citations.

2.12 Comments (SHOULD)

• Implement persistent threaded comments on a submission, exact version or finding.

• Store author, role, created/updated timestamps, parent comment and visibility.

• Support replies, read state if included, and permission-controlled edit/delete.

• Reloaded data must come from the database and be consistent across devices.

2.13 Governance Dashboard (SHOULD)

• Use actual database data to calculate submissions, status distribution, approvals, requests for changes, common issue types, high-severity findings, average review time and trends.

• Include “rules that may be unclear” based on measurable dismissals/reasons, with minimum sample safeguards.

• Add filters by date, brand, channel and role where appropriate.

• No hard-coded charts or demo counters in production.

2.14 Accessibility Review (LATER, implement honest MVP)

• Implement reliable text-level checks: missing image alt text, vague link text where available, readability warnings, excessive all-caps and media caption/transcript reminders.

• Mark checks as automated suggestions, not a full WCAG certification.

• Add manual reviewer checklist for checks not technically verifiable from the submitted text.

2.15 Visual Brand Checker (LATER, implement honest MVP)

• Store approved logos and visual assets with metadata.

• Provide a side-by-side manual comparison workspace and palette guidance.

• If a real vision provider is configured, add evidence-grounded visual observations.

• If no vision capability exists, label the process manual; do not fake logo recognition, pixel-perfect checks or automatic colour compliance.

3. REAL AUTHENTICATION AND ORGANISATION SECURITY

Replace demo persona identity with secure authentication:

• Implement organisation registration with first admin, login, logout, current-user endpoint, password reset or a clearly documented secure alternative, token/session expiry and revocation.

• Hash passwords with a modern password hashing algorithm (Argon2id or bcrypt with appropriate parameters).

• Prefer secure HTTP-only, Secure, SameSite cookies for browser sessions, with CSRF protection where applicable; if using bearer JWT, use short-lived access tokens and a safe refresh/revocation design.

• Validate token signature, issuer, audience, expiry and claims server-side.

• Remove acceptance of raw user ID, email or role as a bearer token.

• Remove `X-Demo-User` from production behavior.

• Do not expose password hashes or secrets in API responses.

• Add rate limits and safe error messages for login.

• Protect last-admin role changes and account deactivation.

• Seed demo users only in local/test mode, with conspicuous demo labels and no production persona switcher.

• Add organisation/brand scoping to every repository query and mutation. Prevent IDOR and cross-organisation access.

4. DATABASE AND PERSISTENCE — MAKE POSTGRESQL REAL

Use PostgreSQL, preferably the planned Supabase PostgreSQL service if credentials are supplied. Add a maintained Node PostgreSQL driver or ORM and create a repository/data-access layer. The Express server must no longer import a mutable in-memory store as its production source of truth.

Tasks:

• Add migrations for all required Round 1 entities and the social-publishing entities below.

• Implement organisations, users, brand_profiles, brand_guidelines, approved_facts, terminology, brand_assets, content_submissions, content_versions, analysis_reports, detected_issues, suggestions, comments, approvals and audit_logs.

• Add social_accounts, social_post_variants, social_variant_versions or equivalent immutable version records, publish_jobs, publish_results and provider_event_logs.

• Add foreign keys, unique constraints, checks, indexes and created/updated timestamps.

• Add organisation_id or an equivalent tenant scope to all tenant-owned records and enforce it in every query.

• Use transactions for version creation, approval decisions, audit writes and publish-job creation.

• Add migrations and development seed scripts. Seed the fictional Lumora example and TS-01 to TS-08 test cases only in development/test environments.

• Replace localStorage business persistence with backend API persistence. Local storage may retain non-sensitive preferences only.

• Make database unavailable errors explicit. Never silently save to localStorage while claiming database success.

• Implement pagination and filtering at database level for large lists.

• Add a readiness endpoint that tests database connectivity and reports degraded state without revealing credentials.

5. COMPLETE THE REST API CONTRACT

Implement and document all API routes required by the actual UI. Route names can follow the existing API contract, but every route must be real, authenticated where appropriate, validated, tenant-scoped and tested.

Authentication and users:

• POST /api/auth/register-organisation

• POST /api/auth/login

• POST /api/auth/logout

• GET /api/auth/me

• POST /api/auth/refresh or the equivalent secure session refresh

• GET /api/users

• POST /api/users

• PATCH /api/users/:id

Brand knowledge:

• GET/POST /api/brand-profiles

• GET/PATCH /api/brand-profiles/:id

• GET/POST /api/brand-profiles/:id/guidelines

• PATCH /api/guidelines/:id

• POST /api/brand-profiles/:id/guidelines/import

• GET/POST /api/brand-profiles/:id/facts

• PATCH /api/facts/:id

• GET/POST /api/brand-profiles/:id/terms

• PATCH /api/terms/:id

• GET/POST /api/brand-profiles/:id/assets

• DELETE/archive routes as appropriate

Submissions and review:

• POST /api/submissions/extract-text for supported file types

• POST /api/submissions

• GET /api/submissions

• GET /api/submissions/:id

• POST /api/submissions/:id/analyse

• PATCH /api/suggestions/:id

• POST /api/submissions/:id/versions

• POST /api/submissions/:id/send-for-review

• GET /api/review-queue

• POST /api/versions/:id/decision

• GET /api/versions/:id/diff

• GET/POST /api/submissions/:id/comments

• PATCH/DELETE comment routes with policy checks

Analytics and audit:

• GET /api/analytics/summary

• GET /api/audit-logs

• POST /api/eval

• GET /api/health and readiness

Social publishing routes are specified in Section 7. Use a consistent response shape, validation schemas, pagination metadata, status codes and machine-readable error codes. Update `docs/API_CONTRACT.md` so it describes only routes that actually exist.

6. AI PIPELINE — SERVER-ONLY, GROUNDED AND AUDITABLE

Follow the Round 1 architecture: React frontend → Express backend → isolated Python FastAPI AI service → Gemini or another swappable provider. A justified single-backend implementation is acceptable only if the AI provider boundary remains isolated and server-only.

• Remove `X-Gemini-Key` support and any browser-supplied AI key override.

• Remove AI provider keys from localStorage, browser requests, client bundles and logs.

• Store provider keys only in server environment secrets or managed secret storage.

• Implement provider abstraction, timeout, bounded retries, rate limiting and usage controls.

• Build context only from the selected brand's active approved guidelines, approved facts, terminology and assets.

• Treat the submitted draft and imported files as untrusted data, not instructions.

• Require structured output validated by a schema.

• Validate that each quoted excerpt exists in the exact submitted version.

• Validate that every cited rule/fact exists in the retrieved context and is active/valid.

• Reject unsupported references or malformed findings.

• Distinguish verified conflict, unsupported claim, terminology, tone, accessibility and needs-human-check.

• Do not let AI approve, publish or silently modify content.

• If external AI is disabled for a brand, run deterministic code checks only.

• If AI is unavailable, show “AI analysis unavailable”; return any valid code-based checks separately. Never invent an AI score or result.

• Persist model/provider, prompt version, reference IDs/versions, confidence, analysis timestamp and failure metadata.

• Provide a labelled evaluation runner based on the Round 1 test set. Report precision/recall or clear pass/fail by case; do not market a small demo test as production accuracy.

7. SOCIAL MEDIA — REAL OAUTH, CHANNEL REVIEW AND APPROVED PUBLISHING

This is a newly required feature. Replace the current simulated connection and fake publishing implementation with genuine official API integrations.

7.1 Providers and capability honesty

• Build provider adapters for official APIs.

• Prioritize Meta Facebook Pages and eligible Instagram Professional accounts, plus LinkedIn organization/member posting where app permissions and provider access permit.

• Support X only when official API access, credentials, pricing/access tier and required permissions are available. Do not claim X integration is live without these.

• Keep provider capabilities in a configuration/metadata layer. Do not advertise features that a provider/account does not support.

• Never collect social account passwords, scrape sites or use browser automation to post.

7.2 OAuth connection centre

• Add official OAuth authorization-code flow with backend callback.

• Validate state and exact redirect URI; use PKCE where supported/required.

• Request minimum necessary scopes and display the scopes granted.

• Resolve eligible pages/organizations/accounts using official provider APIs.

• Store external account IDs returned by the provider, not generated placeholders.

• Show connected, expired, revoked, permission-limited, disconnected and error states.

• Provide reconnect, disconnect and revocation handling.

• Keep provider client secrets and access/refresh tokens server-side only.

7.3 Token security

• Encrypt tokens at rest using a server-side key or managed secret service.

• Never return raw tokens to the frontend or log them.

• Refresh tokens safely when provider supports refresh.

• Redact secrets from exceptions and diagnostics.

• Store token expiry and last verification timestamps.

• On revoked/expired access, block publishing and request reconnect.

7.4 Channel-specific content variants

• From an approved source submission, let an authorized user create a channel variant for a connected account.

• Store caption/body, hashtags, link, assets, alt text where supported, channel metadata and target account.

• Start from source content but require explicit review of channel-specific modifications.

• Each variant has immutable versions and its own analysis and human approval.

• Any edit to caption, hashtags, link, asset or publishable metadata after approval invalidates that variant's approval and requires re-analysis and fresh approval.

• Approval must be attached to exact variant version ID and target account/provider.

• Do not set approval true by default.

• Before publishing, re-check exact version approval, account connection, scopes, token validity, asset availability and provider constraints.

7.5 Publish and schedule

• Provide Publish Now and Schedule for Later.

• Scheduling must include timezone and store canonical UTC execution time plus the original timezone context.

• Use a background worker/queue so a browser tab is not required.

• Use idempotency keys and unique constraints to prevent duplicate posts.

• Track queued, scheduled, publishing, published, failed, cancelled, needs_reconnect and verification_pending states.

• Use bounded exponential retry only for transient errors. Do not retry permanent provider validation or permission failures endlessly.

• Save provider post ID, canonical URL when available, confirmed publish time, target account, exact variant version, initiating user and sanitized provider response.

• Do not mark published until the provider confirms success.

• If provider outcome is uncertain, reconcile via provider API/webhook before retrying; show verification pending rather than claiming success.

• Support cancellation only before execution has begun, with race-safe worker handling.

• Add provider webhook signature verification if webhooks are used.

7.6 Social publishing UI

• Social Accounts page: provider, external account, page/organization, scopes, status, connected owner, expiry and last verification.

• Variant editor: channel preview, character/media constraints, caption, hashtags, link and asset selection.

• Variant analysis: findings and evidence from brand rules/facts.

• Approval screen: exact variant version, exact target account and reviewer decision.

• Publish confirmation: exact text/media/account/time before execution.

• Schedule list/calendar: upcoming jobs and timezone.

• Publishing history: confirmed provider post URL/ID, timestamp, exact version and actor.

• Failure detail: provider-safe error, retry eligibility and reconnect action.

• Do not use hard-coded sample hashtags as the default for every brand.

7.7 Social API routes

Implement equivalents of:

• GET /api/social/providers

• GET /api/social/accounts

• POST /api/social/connect/:provider/start

• GET /api/social/connect/:provider/callback

• DELETE /api/social/accounts/:id

• POST /api/submissions/:id/social-variants

• GET /api/submissions/:id/social-variants

• POST /api/social-variants/:id/versions

• POST /api/social-variants/:id/analyse

• POST /api/social-variants/:id/submit-for-approval

• POST /api/social-variants/:id/approve

• POST /api/social/publish

• POST /api/social/schedule

• GET /api/social/publishing

• GET /api/social/publishing/:id

• POST /api/social/publishing/:id/cancel

• POST /api/social/publishing/:id/retry

Adapt naming to the existing conventions but preserve all behavior and authorization requirements.

8. COMMENTS, APPROVALS, VERSIONING AND AUDIT INTEGRITY

• Comments must reference a submission and optionally an exact content version or finding. Thread replies must reference a valid parent comment in the same authorized context.

• Approval records must include reviewer, decision, exact version ID, reason/comment and timestamp.

• A version cannot be edited after creation. New content always creates a new version.

• A source submission approval does not imply approval of a social variant.

• A social variant approval is tied to a specific variant version, provider and account.

• Publishing result must retain the exact variant version ID and content snapshot that was sent.

• Audit logs are append-only for ordinary users and cover user/role changes, brand/guideline/fact/term changes, submissions, analysis, suggestions, comments, version creation, approval decisions, social OAuth, token revocation, scheduling, cancellation, retries and provider results.

• Use transactions so the business action and audit event are committed together.

• Record actor, organisation, action, entity type/ID, timestamp, previous/new state and sanitized metadata.

9. FRONTEND, DESIGN AND ACCESSIBILITY

Preserve the existing INKLOOM visual style, typography, palette and page structure where practical. Avoid a wholesale redesign. Complete the existing components and make the experience consistent.

• Connect every visible control to a real API action.

• Remove fake success notifications and placeholder values in production.

• Provide loading, empty, validation, success, permission-denied, provider-disconnected and error states.

• Add clear status labels and explanatory messages.

• Ensure responsive layouts on desktop, tablet and mobile.

• Use semantic headings, labels, keyboard navigation, visible focus, accessible dialogs, proper form errors and sufficient contrast.

• Do not communicate state through colour alone.

• Keep demo content visibly labelled and ensure it cannot be confused with real organisation data.

• Make social preview reflect the exact selected platform and variant.

• Ensure no dead buttons, inert tabs or fake chart controls remain.

10. FILES, ASSETS, DEPLOYMENT AND CONFIGURATION

• Use private object storage (Cloudinary or another configured provider) for brand assets and social media media. Validate file type, size and ownership. Store safe metadata and private references.

• Use short-lived signed URLs where private assets need to be accessed by a worker/provider.

• Add `.env.example` with placeholders only for database URL, auth/session secrets, AI provider key, token encryption key, OAuth client IDs/secrets, callback URLs, object storage and job queue configuration.

• Never commit actual credentials.

• Configure CORS allowlists, secure headers, request size limits, rate limits, CSRF protections where relevant and production cookie settings.

• Deploy frontend and backend with correct API base URLs and server-side secrets. Deploy worker as a continuously running service.

• Add database migration command and documented rollback/backup approach.

• Provide provider setup instructions including app registration, redirect URI, scopes, review requirements and test-account steps.

• Provide a production checklist for HTTPS, secrets, DB backups, monitoring, error reporting and worker health.

11. TESTING — REQUIRED END-TO-END ACCEPTANCE

Write and run unit, integration and end-to-end tests. At minimum, prove these scenarios:

AUTH AND TENANCY

1. Unauthenticated requests to protected endpoints return 401.

2. A creator cannot call reviewer/admin endpoints.

3. A user cannot read or mutate another organisation's brand, submission, comments or social account by changing an ID.

4. Raw user ID, email, role or X-Demo-User cannot authenticate a request.

5. Demo persona switching is disabled in production.

DATABASE

6. Create/update data, restart the server and confirm the data remains.

7. Two browser sessions/devices see the same saved submissions, comments and approvals.

8. Database outage returns a clear error and never reports a false successful save.

ROUND 1 WORKFLOW

9. Admin creates a brand, guideline, fact, term and asset.

10. Creator submits content and version 1 is persisted.

11. Analysis returns only references that exist and exact excerpts from the version.

12. Terminology, approved fact, unsupported claim and voice checks produce the expected labels.

13. AI provider failure shows unavailable status and preserves deterministic checks.

14. Accepting/editing a suggestion creates a new immutable version and new analysis.

15. Creator cannot approve their own submission.

16. Reviewer can request changes or approve the exact version.

17. Any content change after approval requires new analysis and review.

18. Comments and replies persist and are permission-scoped.

19. Analytics and audit logs are computed from real records.

SOCIAL PUBLISHING

20. OAuth state mismatch is rejected.

21. Invalid/expired/revoked provider tokens block publishing.

22. Provider tokens never appear in browser storage, frontend responses or logs.

23. A manually typed fake account cannot be marked connected.

24. Unapproved source submission cannot create a publishable approved state.

25. A changed social variant cannot be published using the source approval.

26. Exact approved variant can be scheduled for the selected connected account.

27. Worker publishes without a browser tab open.

28. Successful provider response records real provider ID and URL where available.

29. Failed provider response is shown accurately and safe retries do not create duplicates.

30. Cancellation and worker execution are race-safe.

31. Provider rate limits and permanent errors are handled differently.

32. Publishing history identifies exact version, account, actor and timestamp.

QUALITY

33. `npm run lint` passes.

34. `npm run build` passes.

35. Backend and worker start successfully with documented configuration.

36. Automated tests pass or list exact external credential blockers.

37. No production route returns hard-coded success for a real external action.

12. IMPLEMENTATION ORDER — FOLLOW THE DEPENDENCIES

Phase 1 — Audit and baseline

• Inspect every source file and existing behavior.

• Create a requirement-to-file matrix.

• Record baseline lint/build/test status.

• Preserve working UI and establish branches/checkpoints.

Phase 2 — Database foundation

• Add PostgreSQL driver/ORM and migrations.

• Build repository layer and seed scripts.

• Replace production in-memory/localStorage business data with database access.

• Add health/readiness and persistence tests.

Phase 3 — Authentication and authorization

• Implement real login/session, roles, tenant scoping and route middleware.

• Remove demo identity headers and unsafe token parsing.

• Add authorization tests.

Phase 4 — Complete Round 1 product

• Implement brand CRUD, guideline versioning, facts, terms, assets.

• Complete submissions, AI analysis, correction, immutable versions, comments, review and approval.

• Complete audit and analytics.

• Implement honest MVP accessibility and visual review.

Phase 5 — AI security and quality

• Move provider integration server-side.

• Validate structured output and evidence references.

• Add deterministic fallback and evaluation tests.

Phase 6 — Social connections and publishing

• Implement OAuth adapters and encrypted tokens.

• Implement channel variants, variant analysis, separate approval and immutable snapshots.

• Implement publish worker, retries, idempotency and history.

• Add provider configuration and honest unconnected states.

Phase 7 — QA and production readiness

• Run all tests and fix failures.

• Test responsive UI and full workflows.

• Configure production deployment, secrets, DB migrations, worker and monitoring.

• Update docs and demo script.

Do not skip ahead to social UI polish while database/authentication foundations remain fake. Do not stop after a phase with TODO placeholders.

13. FINAL DELIVERABLES AND HANDOVER

Return the completed existing project with:

1. All source-code changes in the existing repository.

2. Working React frontend, Express backend and isolated AI service or documented equivalent.

3. PostgreSQL migrations, repository layer and development seed data.

4. Secure authentication and server-enforced role/organisation permissions.

5. All Round 1 features implemented to the scope above.

6. Real official social OAuth adapters and approved publishing workflow.

7. Background scheduling worker and publish history.

8. Updated API contract and architecture diagram/notes.

9. `.env.example` with placeholders only.

10. Local setup and production deployment guide.

11. Social provider setup guide with scopes, callback URLs and limitations.

12. Unit, API/integration and end-to-end tests.

13. Test report with commands run, pass/fail counts and external blockers.

14. Feature completion matrix: Implemented / Partially implemented / Blocked, with evidence (route, component, migration and test names).

15. Updated demo script that demonstrates: organisation login → brand setup → guideline/fact/term setup → draft submission → AI analysis → correction → new version → reviewer approval → social OAuth connection → channel variant → analysis → variant approval → schedule/publish → publishing history and audit log.

The final handover must state:

• What was changed and in which files.

• Which commands were run.

• Whether lint/build/tests pass.

• How to configure database, AI provider and social providers.

• Which social providers were actually tested with real credentials.

• What is still blocked and why.

• Exact steps to run locally and deploy.

Do not report a feature as complete based on a screenshot or a route stub. Do not fabricate test results, API responses, provider post IDs, account connections or deployment success.

14. NON-NEGOTIABLE RULES

• Modify the supplied project; do not replace it with a disconnected application.

• Use the Round 1 proposal as the source for the original product scope and priorities.

• Implement the user's new social publishing requirement as an added scope item.

• Preserve the product's core promise: the AI checks and explains; humans decide.

• Database is the source of truth for shared business data.

• Authentication and permissions are enforced on the server.

• AI provider keys and social tokens remain server-side.

• Every content and social variant version is immutable.

• Approval belongs to an exact version.

• Publish only the exact channel-specific version that a human approved.

• Any post-approval change requires re-analysis and re-approval.

• Use official provider APIs and OAuth; never collect passwords or scrape.

• Never fake a save, analysis, approval, account connection or publish success.

• Clearly label unavailable provider access and manual-only checks.

• Run tests and provide evidence before declaring completion.

FINAL COMMAND

Start by reading the current ZIP, Round 1 proposal and this prompt. Inspect the existing implementation and reproduce the audit findings against the actual files. Then make the changes in the repository in the implementation order above. Keep the application runnable after each phase. Complete the backend, database, security, workflow, AI and social publishing—not just the screens. Run the required checks, fix the failures, and deliver the final working project with honest evidence and setup instructions.